

Frontend Architecture, State & Validation Plan

React + TypeScript implementation blueprint for a robust four-step wizard

1. Recommended stack

Concern	Choice	Why
Build	Vite + React + TypeScript	Fast local iteration, simple assessment deployment.
Styling	Tailwind CSS + CSS variables	Precise responsive parity with a small token layer.
Forms	React Hook Form	Low re-render form state and mature field handling.
Validation	Zod	Single typed validation source; easy cross-field refinements.
Wizard state	Typed Context + useReducer or Zustand	Keeps step data when navigating backward without overengineering.
Notifications	Sonner or lightweight custom toast	Global simulated submission failures.
Icons	Lucide React or exact reference assets	Consistent SVG rendering; use exact assets when fidelity demands it.

2. State model

Keep server-like state and UI state separate even though the exercise is frontend-only. A single onboarding draft object should contain normalized field values; step completion should be derived from validation rather than manually toggled booleans.

currentStep + draft + verificationStatus + requestStatus + lastError is enough for most of this assignment.

Suggested state transitions

1. LANDING → TERMS or SIGNUP_START according to reference behavior.
2. SIGNUP_STEP_1 → VERIFYING → VERIFIED or ERROR.
3. VERIFIED → STEP_2 → STEP_3 → STEP_4.
4. Any later step → BACK without deleting valid draft data.
5. STEP_4 → SUBMITTING → SUCCESS or ERROR.
6. Retry from ERROR without duplicating previous request effects.

3. Validation strategy

Validation	When	Example behavior
Required	blur + submit	Whitespace-only values normalize to empty and fail.
Email	change after touched + submit	Trim, validate format, preserve user-friendly error.
Age/year	change/blur	Digits only; sane bounds; explicit under-18 handling if applicable.
Phone if present	change/blur	Digits/allowed prefix only; do not

		coerce invalid strings silently.
Character limits	input	Prevent pathological length and show remaining count only if useful.
Cross-field	dependent field change	Changing state clears an incompatible city/college selection.
OTP	completion + submit	Fixed length, numeric/alphanumeric exactly as reference requires.

4. Mock API contract

Because the brief says frontend only, create a deterministic mock service rather than fake backend complexity. Use short delays so loading states are visible in the recording.

- `verifyEmail(email)`: resolves success for normal inputs; use a documented demo trigger such as `fail@example.com` to simulate a failure.
- `verifyOtp(code)`: use one documented local demo code, e.g. `123456`, and reject other codes after a short delay.
- `submitProfile(payload)`: simulate success/failure; never call a real service with personal data.
- Use `AbortController` or request identity guards so stale async results cannot overwrite a newer action.

5. Folder structure

```
src/
  app/ (router, providers)
  pages/ (Landing, Terms, Signup, Success)
  features/onboarding/ (Wizard, steps, schemas, store, mockApi)
  components/ui/ (Button, Field, Select, OTP, Toast, Progress)
  styles/ (tokens.css, globals.css)
  assets/
  test/
```

6. Rendering and performance rules

- Keep each step isolated so typing in one field does not rerender the entire application.
- Memoize only measured hotspots; do not scatter `React.memo/useMemo` everywhere.
- Prefer uncontrolled inputs through React Hook Form.
- Lazy-load only route-level or heavy decorative assets; do not lazy-load tiny critical controls.
- Use stable keys and derived selectors for dependent options.
- Never put the full form object into URL query parameters.